AEGIS-AGENT HACKATHON KIT

OPERATOR RUNBOOK & PITCH DECK

AEGIS-AGENT
Multi-Agent Security War Room

Operational demonstration guide, architectural breakdown, and complete judge presentation runbook for the autonomous cybersecurity workflow prototype.

> **Persistent Security Invariant:** Prototype for explicitly authorized local testing only. Arbitrary public scanning, automated repository writing, and destructive exploit execution are strictly disabled. All candidate code changes require human-in-the-loop approval.

LOCAL HOST APPLICATION
http://localhost:3000

BACKEND ARCHITECTURE
Zero-dependency Native Node.js HTTP Server

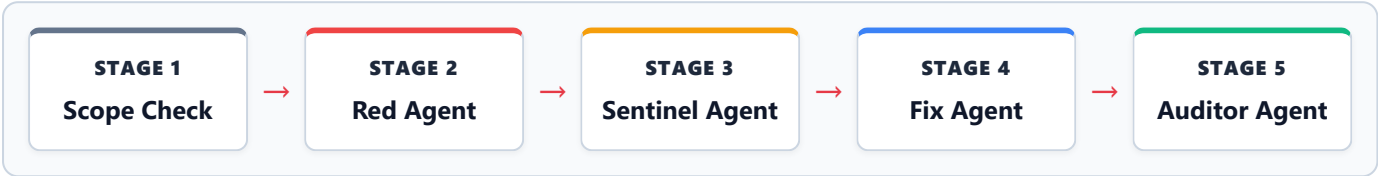
WORKFLOW STAGES
5 Stages (Scope ➔ Red ➔ Sentinel ➔ Fix ➔ Auditor)

TEAM BOUNDARY
Isolated Fix Agent Adapter Interface

01. Executive Summary & Core Value Proposition

Modern application security is plagued by fragmented workflows: security scanners dump hundreds of raw vulnerability alerts onto developers with zero contextual evidence, while automated AI patching tools frequently introduce syntax errors, break API contracts, or write untested code directly to production without human validation.

AEGIS-AGENT solves this by decomposing security remediation into a **4-agent collaborative war room** governed by a strict **Workflow Coordinator** and **Human-in-the-Loop approval gate**:



The 5-Stage Roles & Responsibilities

Stage	Responsible Agent	Function & Safety Boundary
1. Scope Check	Scope Controller	Enforces target allowlist and explicit operator scope consent. Rejects external domains immediately.
2. Red Agent	Controlled Prober	Executes only non-destructive, allowlisted CWE test probes against local container fixtures.
3. Sentinel Agent	Evidence Reviewer	Verifies raw HTTP response signatures. Findings are passed to Fix Agent *only* if verified.
4. Fix Agent	Defensive Synthesizer	Synthesizes candidate patch in isolated memory; runs allowlisted tests (e.g. `py_compile`). Never auto-applies.
5. Auditor Agent	Independent Verifier	Executes 4-check regression matrix. Strictly independent of Fix Agent internal checks.

02. How to Run All Functions in the Project

1. Server Startup

```
# Start the unified backend API & frontend server (Port 3000):
npm start
# OR
node server.js
```

Open browser at: **http://localhost:3000**

2. Keyboard Shortcuts Cheat Sheet

Key	Function	When to Use During Demo
`N`	**New Demo Run**	Press anytime to open the run configuration modal.
`Space`	**Pause / Resume**	Pause simulation to highlight a specific agent's live action to judges.
`R`	**Reset Active Run**	Quickly reset current run back to Stage 1.
`Esc`	**Close Modal**	Dismiss active dialogs instantly.
`1 – 4`	**Switch Tabs**	1 Finding, 2 Proposed Fix, 3 Regression Checks, 4 Logs.

3. The Three Demo Scenarios

Scenario A: Golden Path — Successful Remediation (Pass) STATUS: COMPLETED

1. Press `N` to open the modal.
2. Select **Input validation check (CWE-89)**.
3. Select **Simulation Workflow Outcome:** `[PASS] Complete`.
4. Check **"I confirm scope authorization"** and click **Start Run**.
5. **Watch the flow:** Red Agent captures SQL error trace → Sentinel confirms (99% confidence) → Fix Agent synthesizes prepared statement diff → Auditor verifies 4/4 checks → Status becomes **COMPLETED**.

Scenario B: Guardrail Path — Sentinel Caution (Needs Review) STATUS: NEEDS REVIEW

1. Press `N` and select **Authentication check (CWE-287)**.
2. Select **Simulation Workflow Outcome:** `[ALERT] Needs Review`.
3. Click **Start Run**.

4. **Watch the safety halt:** Sentinel detects ambiguous reflection. The pipeline **halts immediately**. Fix Agent and Auditor remain queued. Proves the system doesn't write code without verified evidence!

Scenario C: Regression Detection — Auditor Flag (Failed) STATUS: FAILED

1. Press `N` and select **Path traversal check (CWE-22)**.
2. Select **Simulation Workflow Outcome:** `[FAIL] Regression Fail`.
3. Click **Start Run**.
4. **Watch independent verification:** Fix Agent proposes a patch, but Auditor's regression suite catches a secondary directory escape bypass. Status becomes **FAILED** (honest reporting!).

4. Human-in-the-Loop Fix Approval

In the **Proposed Fix (Diff)** tab (2), click **"Approve for Demo"**. The server updates the run state to `APPROVED_FOR_DEMO` and turns the badge emerald. Explain to judges: "*Approval records human operator consent; it never silently writes code to disk or pushes to GitHub.*"

5. One-Click Audit Report Download

In the **Dashboard & History** tab, click **"Export Audit Report (JSON)"**. The browser downloads a complete, sanitized compliance report (`AEGIS-Audit-Report-RUN-XXXX.json`) with all secret credentials redacted.

03. Pitch & Presentation Script for Judges

The 30-Second Elevator Pitch

> **SPEAKER:**
> *"Judges, traditional DevSecOps tools overwhelm developers with noisy alerts, while generative AI coding bots are dangerous because they write unverified code straight to production. We built AEGIS-AGENT: a multi-agent security war room that models security operations with specialized agents—Red, Sentinel, Fix, and Auditor. Most importantly, it enforces human approval before any change can be applied, and independently audits regression tests so the bot never grades its own homework."*

The 2-Minute Live Demo Script (Step-by-Step)

Time & Action	What You Say to the Judges	What the Judges See on Screen
0:00 – 0:25 Show Dashboard	*"Here is the AEGIS War Room. Notice the Red & Black HUD displaying our multi-agent roster and persistent safety notice: this prototype is strictly bounded to authorized local applications."*	Sleek cyber dashboard, live telemetry feed, multi-agent roster, UTC clock, and persistent local testing notice banner.
0:25 – 0:50 Press `N` & Launch Run	*"I'll launch a security run for SQL parameter boundary injection. Notice our strict scope confirmation: external URLs are completely rejected by our backend allowlist."*	Modal opens, user confirms authorization checkbox, clicks Start Run. Progress bar begins sweeping with millisecond stopwatch.
0:50 – 1:20 Show Evidence (1) & Diff (2)	*"Red Agent executed the probe. Sentinel verified the database syntax error. Now Fix Agent synthesized a minimal parameterized query fix and verified it with allowlisted python compilation."*	Raw HTTP request/response tabs; color-coded Unified diff viewer with `+6 / -4` line additions/deletions.
1:20 – 1:45 Click Approve & Show Auditor (3)	*"Notice the patch is NOT automatically applied. It requires my approval. Once approved, the Auditor Agent independently runs a 4-check regression suite to prove the bug is remediated."*	Clicking 'Approve for Demo' turns badge green. Auditor displays 4 green verification badges: `REMEDIATED — Not Reproducible`.
1:45 – 2:00 Export Report	*"Finally, with one click, we generate an auditable, sanitized JSON report ready for compliance and compliance officer sign-off. That's AEGIS-AGENT."*	File `AEGIS-Audit-Report-RUN-1050.json` downloads instantly in browser.

Q&A Defense Guide: Winning Answers to Tough Questions

Q: Did your Red Agent actually execute real live network exploits?

Answer: "No, and that is by design for safety and determinism. The Red Agent operates in a controlled sandbox executing predefined, non-destructive test probes against our authorized local test fixture. We strictly reject arbitrary targets and external internet hosts."

Q: Does your Fix Agent rewrite code directly in the user's Git repository?

Answer: "Absolutely not. In enterprise security, unattended code modification is a major risk. Our Fix Agent synthesizes candidate diffs in an isolated memory sandbox and runs allowlisted syntax validation. Human operator approval is strictly required before any change can be applied."

Q: Why not just use a single LLM prompt instead of 4 separate agents?

Answer: "Separation of concerns is critical for security. If the agent that generates the fix also verifies the fix, it suffers from self-confirmation bias. By separating Sentinel (evidence triage), Fix (remediation), and Auditor (independent regression testing), each agent has an isolated, auditable role."

Q: How does your backend connect to the frontend?

Answer: "Our backend is a native, zero-dependency Node.js HTTP server. It serves both the static UI and the REST API (`/api/runs`) on the exact same port (3000), eliminating CORS issues and ensuring 100% offline hackathon reliability without external npm packages."